



UNIVERSIDAD DE GRANADA



Tecnologías Web

Grado en Ingeniería Informática

Programación de aplicaciones en el servidor con PHP (1)

El lenguaje de programación PHP

Este documento está protegido por la Ley de Propiedad Intelectual (Real Decreto Ley 1/1996 de 12 de abril). Queda expresamente prohibido su uso o distribución sin autorización del autor.

(C) Javier Martínez Baena
jbaena@ugr.es
 Departamento de Ciencias de la Computación e Inteligencia Artificial
<http://decsai.ugr.es>



Tecnologías Web

3º Grado en Ingeniería Informática

PHP (1). Introducción a PHP

»»

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP(1). Introducción al lenguaje

¿Por qué es necesaria?

¿Por qué hace falta la programación en el lado del servidor? (BACK-END)



```

graph LR
    Clientes[Clientes navegadores] -- "Petición de URL" --> Servidor[Servidor Web]
    Servidor -- "Si es .html, .jpg, ..." --> FS[Acceso a FS]
    FS -- "Recupera el fichero y lo devuelve" --> Servidor
    Servidor -- "Si es .php (u otro lenguaje)" --> Interpretador[Interprete del lenguaje]
    Interpretador -- "HTML, .jpg, ..." --> Servidor
    Interpretador --> BBDD[Acceso a BBDD]
    Interpretador --> Calculo[Calculo]
    
```

Procesamiento en el servidor:

- Generación de código **HTML dinámico** en el servidor
- Acceso a recursos de almacenamiento: **Bases de Datos**, ficheros
- Acceso a recursos de cálculo: **CPU**
- Acceso a otros recursos
- Comunicación con **otros servicios**: POP3, IMAP, HTTP, ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

3

PHP(1). Introducción al lenguaje

¿Por qué PHP?

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

PHP(1). Introducción al lenguaje

¿Por qué PHP?

W3Techs <https://w3techs.com/>

Web Technology Surveys

Language	Percentage
PHP	76.6%
ASP.NET	6.5%
Ruby	5.6%
Java	4.7%
JavaScript	3.2%
Scala	3.0%
static files	1.8%
Python	1.4%
ColdFusion	0.3%
Perl	0.1%
Erlang	0.1%

Percentages of websites using various server-side programming languages
Note: a website may use more than one server-side programming language

W3Techs.com, 17 January 2024

Análisis de top 10.000.000 sites (Enero 2024)
Lenguajes en el servidor

Server-side Programming Languages, Market Positions, W3Techs.com, 17 Jan 2024

used by high traffic sites

used by low traffic sites

used by fewer sites

used by many sites

PHP

Version	Percentage
Version 7	59.1%
Version 8	23.6%
Version 5	17.2%
Version 4	0.2%

Percentages of websites using various versions of PHP

W3Techs.com, 17 January 2024

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

PHP(1). Introducción al lenguaje

Historia de PHP

PHP = "PHP: Hypertext Preprocessor"
(inicialmente "Personal Home Page Tools")

Creado por Rasmus Lerdorf en 1994

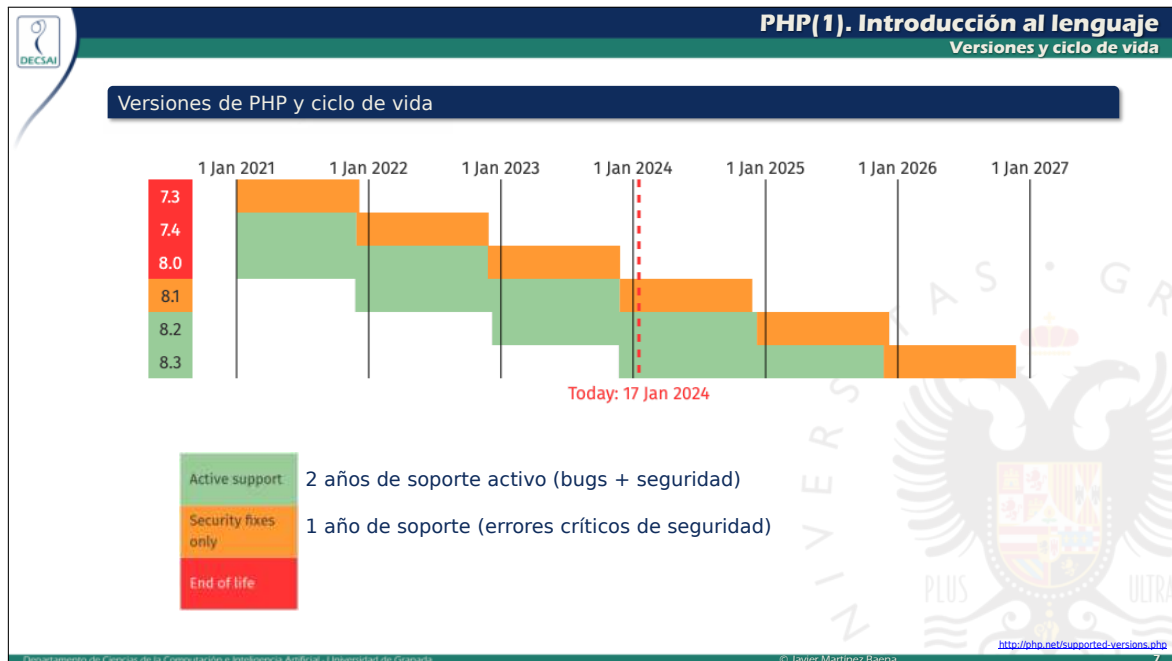
- PHP 1.- 1995. Sale primera versión
- PHP 2.- 1997
- PHP 3.- 1998.
Renombrado a PHP: Hypertext Preprocessor.
Parser reescrito por Zeev Suraski y Andi Gutmans.
Zend Engine = núcleo del intérprete PHP
- PHP 4.- 2000
- PHP 5.- 2004. Incluye POO
- PHP 6.- Intento de dar soporte completo Unicode. Abandonado
- PHP 7.- 2014. Mejoras de rendimiento
- PHP 8.- 2020

<http://php.net/manual/es/history.php.php>
<https://en.wikipedia.org/wiki/PHP>

No hay una especificación formal
Desarrollado por The PHP Group (<http://www.php.net>)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP(1). Introducción al lenguaje

Marcas de PHP

Cómo escribir código PHP

- Fichero con extensión .php
- Inclusión de código entre las marcas `<?php ?>`

hola.php

```
<?php
echo "Hola mundo";
?>
```

El fichero puede incluir HTML y PHP "mezclado" ⚠

hola2.php

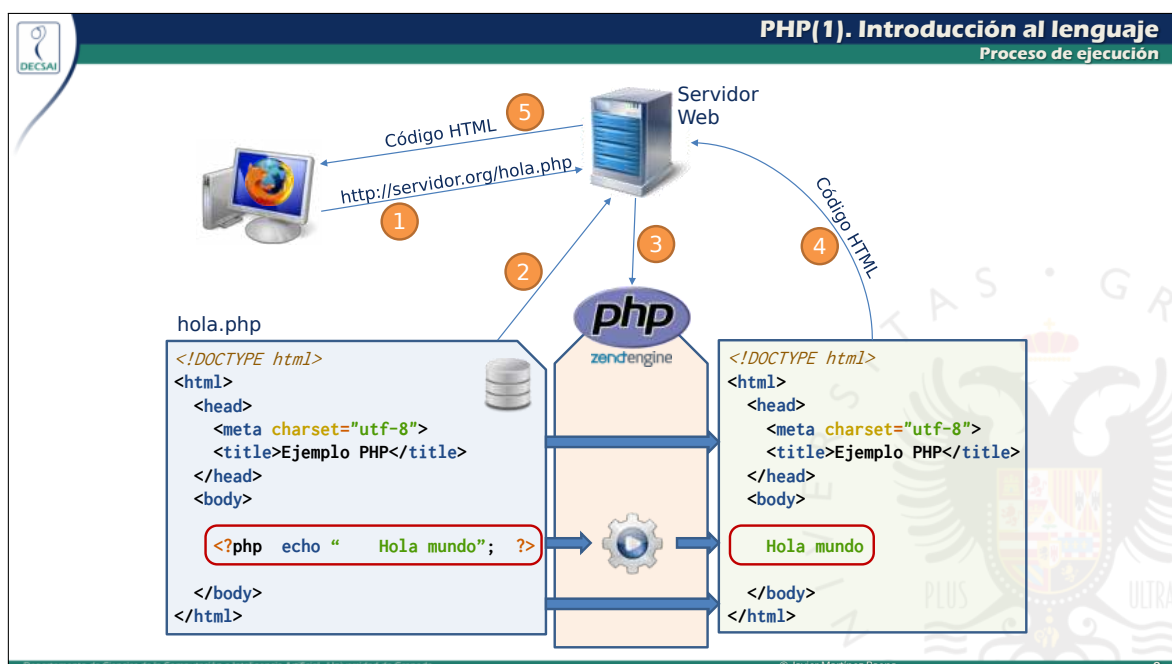
```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Ejemplo PHP</title>
</head>
<body>
<?php echo "Hola mundo\n"; ?>
</body>
</html>
```

Documento devuelto

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Ejemplo PHP</title>
</head>
<body>
  Hola mundo
</body>
</html>
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP(1). Introducción al lenguaje

Marcas de PHP

Mezclando PHP y HTML: procurar desacople y legibilidad

1

```
<body>
<ul>
<?php for ($i = 1; $i <= 5; $i++) { ?>
<li><p>Párrafo número <?php echo $i; ?></p></li>
<?php } ?>
</ul>
</body>
```

```
<body>
<ul>
<li><p>Párrafo número 1</p></li>
<li><p>Párrafo número 2</p></li>
<li><p>Párrafo número 3</p></li>
<li><p>Párrafo número 4</p></li>
<li><p>Párrafo número 5</p></li>
</ul>
</body>
```

2

```
<body>
<?php
echo "<ul>\n";
for ($i = 1; $i <= 5; $i++)
echo "<li><p>Párrafo número $i </p></li>";
echo "</ul>\n";
?>
</body>
```

```
<body>
<ul>
<li><p>Párrafo número 1</p></li><li><p>Párrafo
número 2</p></li><li><p>Párrafo número
3</p></li><li><p>Párrafo número 4</p></li>
<li><p>Párrafo número 5</p></li>
</ul>
</body>
```

1 Ligeramente más eficiente que 2 (poca diferencia)
Mejor edición (coloreado de tags, tabulaciones, cierres, etc)

2 Más legible el código PHP / menos legible el código HTML
Formateo de salida HTML incluido en cadenas PHP

Criterio para decidir:
legibilidad, extensión, ...

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 10

PHP(1). Introducción al lenguaje

Inclusión de ficheros

Inclusión de código de otros ficheros

Hay dos instrucciones para incluir código de otros ficheros:

- `require "fichero"` Si el fichero no existe provoca error
- `include "fichero"` Si el fichero no existe provoca solo warning

Si se incluye un mismo fichero múltiples veces el código queda duplicado ... solución:

- `require_once`, `include_once` : para que la segunda inclusión no tenga efecto

```
<?php include "header.html"; ?>
<h1>El título</h1>
<?php include "footer.html"; ?>
```

header.html

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
</head>
</body>
```

footer.html

```
<footer>
<p>(C) Pepito Pérez</p>
</footer>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
</head>
<body>
<h1>El título</h1>
<footer>
<p>(C) Pepito Pérez</p>
</footer>
</body>
</html>
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 11

PHP(1). Introducción al lenguaje

Inclusión de ficheros

main.php

```
<?php
require "func_html.php";
htmlIni("Ejemplo");
echo "Hola mundo";
htmlFin();
?>
```

```
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Ejemplo</title>
</head>
<body>
Hola mundo
</body>
</html>
```

func_html.php

```
<?php

function htmlIni($titulo) {
    echo '<!DOCTYPE html> <html> <head> <meta charset="utf-8">';
    echo "<title>$titulo</title> </head> <body>";
}

function htmlFin() {
    echo '</body> </html>';
}

?>
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 12



PHP(1). Introducción al lenguaje

Carga de ficheros

Lectura de datos

```
$cad = file_get_contents($nombre)
```

Carga el contenido el fichero y lo devuelve como una cadena
El nombre puede ser una URL
Si falla la lectura devuelve FALSE

```
$cad = file_get_contents("datos.txt");
echo "Tamaño: ", strlen($cad), PHP_EOL;
echo "Cadena: ", $cad, PHP_EOL;
```

```
$vec = file($nombre)
```

Carga el contenido el fichero y lo devuelve como un vector de cadenas. Cada elemento del vector es una línea del fichero.
Si falla la lectura devuelve FALSE

```
$cads = file("datos.txt");
foreach ($cads as $c)
    echo "L (" , strlen($c), "): " , $c;
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
13



PHP(1). Introducción al lenguaje

Carga de ficheros

Lectura de datos

```
readfile($nombre)
```

Lee el contenido el fichero y lo envía a la salida
El nombre puede ser una URL
Devuelve el número de bytes leídos o FALSE

```
hola.html
<!DOCTYPE html>
<html> <head>
<meta charset="utf-8">
<title>Tecnologías Web</title>
</head> <body>
<h1>Hola mundo</h1>
</body> </html>
```

```
readfile("hola.html");
```

// ... equivale a:

```
$cad = file_get_contents("hola.html");
echo $cad;
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
14



PHP(1). Introducción al lenguaje

Aspectos genéricos

Algunos aspectos básicos

- Comentarios:
 - // (comentarios estilo C++)
 - /* */ (comentarios estilo C)
 - # (comentarios estilo Perl)
- Usa punto y coma para separar instrucciones
- Case sensitive:
 - Las variables SÍ son sensibles a mayúsculas
 - Las funciones, clases y palabras reservadas NO lo son

Mostrar mensajes en salida

- Instrucción echo (o print)
- Salto de línea: "\n" o PHP_EOL → (Útil para ejecución en línea de órdenes)
- printf()

```
<?php echo "Hola mundo", PHP_EOL;
echo "Hola mundo\n";
echo "Hola ", "mundo", PHP_EOL; // Múltiples argumentos
print "Hola mundo\n";           // Solo un argumento
echo("Hola mundo\n");           // También con paréntesis
print("Hola mundo\n"); ?>
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
15



UNIVERSIDAD
DE GRANADA

Tecnologías Web

3º Grado en Ingeniería Informática

PHP (1). Introducción a PHP

1. Introducción
- » 2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena



PHP(1). Variables y tipos

Variables y tipos

Tipos

- **Tipificación débil**

Escalares - Enteros - Reales - Cadenas - Booleanos	Compuestos - Arrays - Objetos	Especiales - Resource - NULL	Constantes predefinidas <pre>PHP_INT_MAX PHP_INT_MIN PHP_INT_SIZE PHP_FLOAT_DIG PHP_FLOAT_EPSILON PHP_FLOAT_MIN PHP_FLOAT_MAX null true false</pre>
--	---	--	--
- Funciones de utilidad:
is_int(), is_float(), is_bool(), is_string(), is_numeric(), ...

Variables

- Los nombres de las variables comienzan por \$
- Las variables son **sensibles a mayúsculas/minúsculas**

<https://www.php.net/manual/es/language.types.php>
<https://www.php.net/manual/en/reserved.constants.php>
<https://www.php.net/manual/es/language.variables.php>
<https://www.php.net/manual/es/ref.var.php>



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

17



PHP(1). Variables y tipos

Datos booleanos y NULL

Datos booleanos

- Literales: true, false (no es sensible a mayúsculas/minúsculas)
- Evalúan a falso estos valores:
 - False
 - 0
 - 0.0
 - ""
 - "0"
 - Array con cero elementos
 - NULL
- Función is_bool() comprueba si el argumento es booleano

El tipo NULL

- Solo existe un valor para este tipo: NULL
- Representa a una variable que no tiene valor
- Función is_null() comprueba si el argumento es NULL



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

18



PHP(1). Variables y tipos

Operadores

Aritméticos	Bit	Asignación	Lógicos (short-circuit)
• ++, -- • *, /, % • +, -	• ~ • &, , ^ • <<, >>	=, +=, -=, *=, **=, /=, %=, .=, ^=, &=, =, <<=, >>=	• ! • and, && • xor • or,

<https://www.php.net/manual/es/language.operators.precedence.php>

Operadores relacionales (todos de igual prioridad)

- ==, !=, <> Igualdad/desigualdad de los valores
- <, >, <=, >= Orden
- ===, !== Igualdad/desigualdad de los valores **y del tipo**

```

2==2 // true
2===2 // true

2==2.0 // true (conversión de tipos)
2===2.0 // false

true==false // false
false==0 // true (conversión de tipos)
false===0 // false

```

Otros operadores

Null coalescing operator, null-safe operator, ... los vemos más adelante

<https://www.php.net/manual/es/language.operators.php>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 19



PHP(1). Variables y tipos

Información de variables

Información sobre variables

var_dump(\$v)	Muestra el tipo y valor de una expresión
print_r(\$v)	Muestra el valor de la expresión
gettype(\$v)	Devuelve el tipo de una variable (como cadena)
isset(\$v)	Comprueba la existencia de una variable

```

$entero = 20;
$cadena = "Texto de prueba";
class miClase {
    public $v1;
    public $v2;
}
$obj = new miClase;
$obj->v1 = 10;
$obj->v2 = [1,2,'hola'];

var_dump($entero);
var_dump($cadena);
var_dump($obj);

print_r($entero);
print_r($cadena);
print_r($obj);

echo gettype($entero); // "integer"
echo gettype($obj); // "object"

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 20



PHP(1). Variables y tipos

Constantes

Constantes

- Definición de constantes
 - define("NOMBRE", "VALOR"); Definición en tiempo de ejecución
 - const NOMBRE = VALOR; Definición en tiempo de compilación
- Recomendación: usar mayúsculas

```

define("CTE1", "valor1");
define("CTE2", true);
define("CTE3", 3);
define(CTE4, 7);

const CTE5 = 6;
const CTE6 = "Hola";

echo CTE4, PHP_EOL;
echo CTE5, PHP_EOL;

```

Constantes predefinidas (Magic constants):

tienen información sobre el código PHP (usadas para depurar):

```

__LINE__
__FILE__
__DIR__
__FUNCTION__
__CLASS__
__METHOD__
__NAMESPACE__

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 21



PHP(1). Variables y tipos
 Conversiones explícitas e implícitas

Conversiones implícitas

- Integer op float : se hace casting a float
- Número op string : el comportamiento varía según versión de PHP 7 / 8

Expresión	PHP 7	PHP 8
0 == "0"	true	true
0 == "0.0"	true	true
0 == "foo"	true	false
0 == ""	true	false
42 == "42"	true	true
42 == "42foo"	true	false
"0" == "0.0"	true	true

PHP 8:
 Si el string es "numérico": se convierte y se compara int/float
 Si no: el número se convierte a string y se comparan
https://wiki.php.net/rfc/string_to_number_comparison
<https://www.php.net/manual/en/language.types.numeric-strings.php>

== hace una comparación "no estricta"
 === hace una comparación "estricta"

Conversiones explícitas

(int)	(integer)	
(bool)	(boolean)	
(float)	(double)	(real)
(string)		

`$x = (int) 4.5; // asigna 4`
`$x = (int)(3 / 2); // asigna 1`

<https://www.php.net/manual/es/language.types.type-juggling.php>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
22



Tecnologías Web
 3º Grado en Ingeniería Informática
PHP (1). Introducción a PHP

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena



PHP(1). Cadenas de caracteres
 Cadenas y literales

Cadenas de caracteres

- Los literales pueden ir entre comillas:
 - " " Dobles: Se sustituye el valor de las variables incluidas y permite el uso de códigos de escape (\n, \t, \, \", ...)
 - ' ' Simples: No se sustituye el valor de las variables incluidas

```

<?php
$nombre = "Javier";
$saluda1 = "Hola $nombre \n ¿Cómo estás?";
$saluda2 = 'Hola $nombre \n ¿Cómo estás?';
echo $saluda1, PHP_EOL;
echo $saluda2, PHP_EOL;
?>
```

```

jbaena@void: ~/public_html/test
jbaena@void:~/public_html/test$ php cadenas2.php
Hola Javier
¿Cómo estás?
Hola $nombre \n ¿Cómo estás?
jbaena@void:~/public_html/test$
```

↑
 Análisis de variables dentro de comillas dobles: **sintaxis simple** vs **sintaxis compleja**

```

$c = "Juan";
echo "El plural de $c es {$c}es";

$d = [[1,2,3],[4,5,6]];
echo "$d[1][2]"; // Error
echo "{$d[1][2]}"; // OK
```

```

$a = [2,3,4];
echo "$a[1]"; // OK

$b = ['nombre'=>'Juan', 'ape'=>'Garrido'];
echo "$b['nombre']"; // Error
echo "{$b['nombre']}"; // OK
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
24



PHP(1). Cadenas de caracteres

Comparación, concatenación

Comparación de cadenas

- Operadores relacionales
 - === y !== no hacen conversión de tipos (comparación estricta)
 - El resto sí hacen conversión de tipos

```

echo ("12"==12 ? "true" : "false"), PHP_EOL; // T Comparación no estricta (numérica)
echo ("12"===12 ? "true" : "false"), PHP_EOL; // F Comparación estricta
echo ("12"=="12" ? "true" : "false"), PHP_EOL; // T Comparación no estricta (numérica)
echo ("12"=="12a" ? "true" : "false"), PHP_EOL; // F Comparación no estricta (texto)
echo (12=="12a" ? "true" : "false"), PHP_EOL; // F Comparación no estricta (texto)
echo ("0"=="0.0" ? "true" : "false"), PHP_EOL; // T Comparación no estricta (numérica)
echo ("0"===0.0 ? "true" : "false"), PHP_EOL; // F Comparación estricta

```

Concatenación

- El operador de concatenación es el punto .
- . = es la concatenación y asignación

```

$nom = "Juan";
$ape = "Garrido";
$completo = $nom . ' ' . $ape;
$segundo = "Padial";
$completo .= ' ' . $segundo;

```

Funciones

PHP dispone de más de 100 funciones para operar con cadenas

<https://www.php.net/manual/es/ref.strings.php>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

25



PHP(1). Cadenas de caracteres

Here documents

Heredoc (operador)

Simplifica la escritura de cadenas de varias líneas de texto (que suele ser código HTML)

```

<<<MARCADOR.␣
...
MARCADOR;␣

```

```

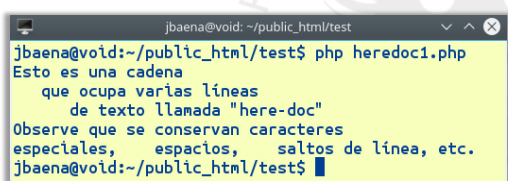
<?php
$palabra = <<<FinTexto
Esto es una cadena
    que ocupa varias líneas
        de texto llamada "here-doc"
Observe que se conservan caracteres
especiales, espacios, saltos de línea, etc.
FinTexto;
echo $palabra;
?>

```



Cuidado:

- Sin sangrado
- Salto de línea
- Acaba en ;



Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

26



PHP(1). Cadenas de caracteres

Here documents

Heredoc

También ayuda a mejorar la legibilidad del documento cuando hay mezcla de PHP y HTML

```

<p>El usuario cuyo login es <?php echo $login; ?> (llamado <?php echo $nombre; ?> <?php echo $apellidos; ?> ) nació en el año <?php echo $fnac; ?> y su cuota de socio es <?php echo $cuota; ?>
</p>

<?php
echo "<p>El usuario cuyo login es $login; (llamado $nombre $apellidos) nació en el año $fnac y su cuota de socio es $cuota</p>";
?>

<?php
echo <<<HTML
<p>El usuario cuyo login es $login (llamado $nombre $apellidos) nació en el año $fnac y su cuota de socio es $cuota</p>
HTML;
?>

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

27



PHP(1). Cadenas de caracteres

Here documents

Heredoc

También ayuda a mejorar la legibilidad del documento cuando hay mezcla de PHP y HTML

```
<div class="usuario">
<p>Los datos del usuario son estos:</p>
<ul class="lista">
<li>Login: <span class="login"><?php echo $login; ?></span></li>
<li>Nombre: <span><?php echo $nombre; ?></span></li>
<li>Apellidos: <span><?php echo $apellidos; ?></span></li>
<li>Año de nacimiento: <span><?php echo $fnac; ?></span></li>
<li>Cuota: <span><?php echo $cuota; ?></span></li>
</ul>
</div>
```

```
<?php
echo "
<div class='usuario'>
<p>Los datos del usuario son estos:</p>
<ul class='lista'>
<li>Login: <span class='login'>$login</span></li>
<li>Nombre: <span>$nombre</span></li>
<li>Apellidos: <span>$apellidos</span></li>
<li>Año de nacimiento: <span>$fnac</span></li>
<li>Cuota: <span>$cuota</span></li>
</ul>
"
?>
```

```
<?php
echo <<<HTML
<div class="usuario">
<p>Los datos del usuario son estos:</p>
<ul class="lista">
<li>Login: <span class="login">$login</span></li>
<li>Nombre: <span>$nombre</span></li>
<li>Apellidos: <span>$apellidos</span></li>
<li>Año de nacimiento: <span>$fnac</span></li>
<li>Cuota: <span>$cuota</span></li>
</ul>
</div>
HTML;
?>
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

28



PHP(1). Cadenas de caracteres

Here documents

Nowdoc (operador)

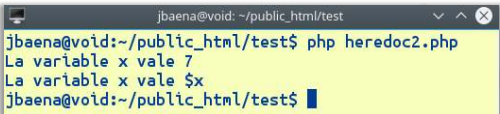
Heredoc es a los strings con comillas dobles lo que Nowdoc es a los string con comillas simples

```
<?php
$x=7;

$palabra1 = <<<FinTexto
La variable x vale $x
FinTexto;
echo $palabra1, PHP_EOL;
```

```
$palabra2 = <<<'FinTexto'
La variable x vale $x
FinTexto;
echo $palabra2, PHP_EOL;
```

```
<?>
```



Observa las comillas simples

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

29



PHP(1). Cadenas de caracteres

Here documents

Heredoc y newdoc: nueva sintaxis más flexible desde PHP 7.3

- Modifica la regla de sangrado (no obliga a cerrar en columna cero)
- No obliga ni al salto de línea ni al punto y coma del cierre

```
<?php
class foo {
    public $bar = <<<EOT
bar
EOT;
}
```

```
<?php
class foo {
    public $bar = <<<EOT
bar
EOT;
}
```

Válido desde PHP 7.3

```
$values = [<<<END
a
b
c
END
, 'd e f'];
```

✓

```
// no indentation
echo <<<END
a
b
c
END;
/*
a
b
c
*/
```

✓

```
// 4 spaces of indentation
echo <<<END
a
b
c
END;
/*
a
b
c
*/
```

✓

```
echo <<<END
a
b
c
END;
```

✗

```
$values = [<<<END
a
b
c
END, 'd e f'];
```

✓ PHP ≥ 7.3
✗ PHP < 7.3

https://wiki.php.net/rfc/flexible_heredoc_nowdoc_syntaxes

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

30



Tecnologías Web

3º Grado en Ingeniería Informática

PHP (1). Introducción a PHP



1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena



PHP(1). Arrays

Arrays

Arrays en PHP

Colección de datos de cualquier tipo

- Indexados. La clave es un entero (comenzando en 0)
- Asociativos key=>value. La clave es un string

Internamente se tratan todos como asociativos y el orden de almacenamiento es el de inserción

```

// Indexados
$a1 = ['Javier', 'Martínez', 'Baena'];
$a2 = array('Javier', 'Martínez', 'Baena');

$a3 = [];
$a3[] = 'Javier';
$a3[] = 'Martínez';
$a3[] = 'Baena';

for ($i=0; $i<3; $i++)
    echo $a3[$i];
        
```

```

// Asociativos
$a4['Nombre'] = 'Javier';
$a4['Apellido1'] = 'Martínez';
$a4['Apellido2'] = 'Baena';

$a5 = ['Nombre'=>'Javier', 'Apellido1'=>'Martínez', 'Apellido2'=>'Baena'];

foreach ($a4 as $e)
    echo $e;

foreach ($a4 as $k=>$v)
    echo $k, $v;
        
```

```

// Internamente son asociativos:
$a6 = [];
$a6[2] = 'Baena';
$a6[0] = 'Javier';
$a6[1] = 'Martínez';

foreach ($a6 as $e)
    echo $e;
    /* BaenaJavierMartínez*/

for ($i=0; $i<3; $i++)
    echo $a6[$i];
    /* JavierMartínezBaena*/
        
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
32



PHP(1). Arrays

Arrays

Arrays multidimensionales

No existen como tal pero ... puede haber *arrays* dentro de *arrays*

```

$b1 = [ ['Javier', 'Martínez', 'Baena'], ['Juan', 'Pérez', 'López'] ];
        
```

```

echo "El nombre es $b1[0][0]"; /* Error */
echo "El nombre es {$b1[0][0]}"; /* OK */
        
```

Funciones sobre arrays

• Hay más de 80 <https://www.php.net/manual/es/ref.array.php>

```

$a1 = ['Javier', 'Martínez', 'Baena'];
$a5 = ['Nombre'=>'Javier', 'Apellido1'=>'Martínez', 'Apellido2'=>'Baena'];

if (in_array('Martínez', $a5))
    echo "Encontrado";

if (array_key_exists('Nombre', $a5))
    echo "Encontrado";
        
```

```

extract($a5);
echo $Nombre, $Apellido1, $Apellido2;

// Para indexados
list($nom, $ape1, $ape2) = $a1;
echo $nom, $ape1, $ape2;

$c1 = compact('nom', 'ape1', 'ape2');
var_dump($c1);
        
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
33



PHP(1). Arrays

Variables superglobales

Variables superglobales: variables globales predefinidas

Variables globales predefinidas

\$GLOBALS
\$_SERVER
\$_GET
\$_POST
\$_FILES
\$_COOKIE
\$_SESSION
\$_REQUEST
\$_ENV

Todas son *arrays* globales disponibles en todos los ámbitos del *script* en ejecución

\$GLOBALS

Incluye todas las variables globales del usuario

```

$vari = 34;
echo $vari;           // 34
echo $GLOBALS["vari"]; // 34

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

34



PHP(1). Arrays

Variables superglobales

Variables superglobales: variables globales predefinidas

\$_SERVER Información creada por el servidor (rutas, cabeceras, etc)

```

$_SERVER["SCRIPT_NAME"]=> "/tw/php/superglobals.php"
$_SERVER["REQUEST_METHOD"]=> "GET"
$_SERVER["QUERY_STRING"]=> ""
$_SERVER["SCRIPT_FILENAME"]=> "/var/www/html/tw/php/superglobals.php"

$_SERVER["HTTP_HOST"]=> "localhost"
$_SERVER["HTTP_USER_AGENT"]=> "Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:52.0)
    Gecko/20100101 Firefox/52.0"
$_SERVER["SERVER_SOFTWARE"]=> "Apache/2.4.18 (Ubuntu)"
$_SERVER["SERVER_NAME"]=> "localhost"
$_SERVER["SERVER_ADDR"]=> "127.0.0.1"
$_SERVER["SERVER_PORT"]=> "80"
$_SERVER["REMOTE_ADDR"]=> "127.0.0.1"
$_SERVER["DOCUMENT_ROOT"]=> "/var/www/html"
$_SERVER["SERVER_PROTOCOL"]=> "HTTP/1.1"
$_SERVER["REQUEST_URI"]=> "/tw/php/superglobals.php"
$_SERVER["PHP_SELF"]=> "/tw/php/superglobals.php"

```

No está garantizado que cualquier servidor disponga de las mismas variables

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

35



PHP(1). Arrays

Variables superglobales

Variables superglobales: variables globales predefinidas

\$_SERVER Información creada por el servidor (rutas, cabeceras, etc)

RFC 3875. The Common Gateway Interface (CGI) 1.1

Según este documento, al menos se debería disponer de:

```

QUERY_STRING
REMOTE_HOST
REQUEST_METHOD
SCRIPT_NAME

```

... y algunas otras menos relevantes

Apache permite configurar si algunas de las variables están o no disponibles en PHP

<http://www.faqs.org/rfcs/rfc3875.html>

No está garantizado que cualquier servidor disponga de las mismas variables

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

36



PHP(1). Arrays
 Variables superglobales

Variables superglobales: variables globales predefinidas

<code>\$_GET</code>	Parámetros pasados al script con el método GET
<code>\$_POST</code>	Parámetros pasados al script con el método POST
<code>\$_REQUEST</code>	Incluye las variables de <code>\$_GET</code> , <code>\$_POST</code> y <code>\$_COOKIE</code>

http://localhost/tw/php/superglobals.php?nombre=Pepito&ape=P%C3%A9rez

```

echo $_GET['nombre']; // Pepito
echo $_GET['ape'];    // Pérez

echo $_REQUEST['nombre']; // Pepito
echo $_REQUEST['ape'];    // Pérez
  
```

<code>\$_FILES</code>	Ficheros subidos por HTTP POST (desde formularios)
<code>\$_COOKIE</code>	Variables pasadas al script mediante HTTP cookies
<code>\$_SESSION</code>	Variables de sesión disponibles
<code>\$_ENV</code>	Variables de entorno del intérprete de PHP

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
37



PHP(1). Arrays
 Variables superglobales

php.ini

Configuración de variables superglobales en el servidor

```

; This directive determines which super global arrays are registered when PHP
; starts up. G,P,C,E & S are abbreviations for the following respective super
; globals: GET, POST, COOKIE, ENV and SERVER. There is a performance penalty
; paid for the registration of these arrays and because ENV is not as commonly
; used as the others, ENV is not recommended on production servers. You
; can still get access to the environment variables through getenv() should you
; need to.
; Default Value: "EGPCS" / ; Development Value: "GPCS" / ; Production Value: "GPCS";
; http://php.net/variables-order
variables_order = "GPCS"

; This directive determines which super global data (G,P & C) should be
; registered into the super global array REQUEST. If so, it also determines
; the order in which that data is registered. The values for this directive
; are specified in the same manner as the variables_order directive,
; EXCEPT one. Leaving this value empty will cause PHP to use the value set
; in the variables_order directive. It does not mean it will leave the super
; globals array REQUEST empty.
; Default Value: None / ; Development Value: "GP" / ; Production Value: "GP"
; http://php.net/request-order
request_order = "GP"
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
38



Tecnologías Web
 3º Grado en Ingeniería Informática
PHP (1). Introducción a PHP

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
- » 5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares


Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena



PHP(1). Funciones
 Definición, ámbito

Funciones

El nombre NO es sensible a mayúsculas. Por convención, suelen nombrarse con minúsculas.

```
function doble($x) {
    return 2*$x;
}
```

Ámbito de variables

Variables definidas fuera de las funciones:

- Son de ámbito global.
- No son accesibles dentro de las funciones.

Variables definidas dentro de las funciones:

- Tienen ámbito local a la función.

Las variables superglobales: tienen ámbito global y local a todas las funciones.

```

$x = 4;
function f() {
    echo $x; // Error
    if (isset($_POST['usuario']))
        echo $_POST['usuario']; // Ok
}
echo $x; // Ok
f(); // Error

```

```

$x = 4;
function f() {
    global $x;
    echo $x; // Ok
}
echo $x; // Ok
f(); // Ok

```

```

$x = 4;
function f() {
    echo $GLOBALS['x']; // Ok
}
echo $x; // Ok
f(); // Ok

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

40



PHP(1). Funciones
 Paso de parámetros

Paso de parámetros

- Por defecto, el paso de parámetros es por valor.
- El paso por referencia se hace anteponiendo & al parámetro formal.

```

function doble($x) {
    $x = 2*$x;
}
$y = 2;
doble($y);
echo $y, PHP_EOL; // 2

```

```

function dobleR(&$x) {
    $x = 2*$x;
}
$y = 2;
dobleR($y);
echo $y, PHP_EOL; // 4

```

Valores por defecto

- Se pueden indicar en la declaración de la función.
- Van al final de la lista de parámetros formales.

```
function dobleD($x=10) {
    return 2*$x;
}
echo dobleD(4), PHP_EOL; // 8
echo dobleD(), PHP_EOL; // 20
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

41



PHP(1). Funciones
 Paso de parámetros

Parámetros variables y llamada

- El número de parámetros puede ser variable.
- Pueden omitirse parámetros en la llamada.
- La llamada puede ser previa a la definición.

```

function sumar() {
    $s=0;
    for ($i=0; $i<func_num_args(); $i++) // Número de argumentos
        $s += func_get_arg($i); // Argumento i-ésimo
    return $s;
}

echo sumar(2,5,4,6), PHP_EOL; // 17
echo sumar(1,3), PHP_EOL; // 4
echo sumar(), PHP_EOL; // 0

function sumar() {
    $s=0;
    $a=func_get_args(); // Array con todos los argumentos
    for ($i=0; $i<count($a); $i++)
        $s += $a[$i];
    return $s;
}

```

```

echo sumar3(2), PHP_EOL;
echo sumar3(2,5,7), PHP_EOL;
echo sumar3(), PHP_EOL;

function sumar3($a,$b,$c) {
    $s=0;
    if (isset($a))
        $s += $a;
    if (isset($b))
        $s += $b;
    if (isset($c))
        $s += $c;
    return $s;
}

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

42



PHP(1). Funciones
 Funciones variables

Funciones variables

La llamada a una función puede hacerse a través del valor de una variable.

```

function suma($a,$b) {
    return $a+$b;
}
function multi($a,$b) {
    return $a*$b;
}

$oper = "suma";
echo $oper(2,4); // 6

$oper = "multi";
echo $oper(2,4); // 8

```

```

// Simplifica estructuras como esta
switch ($oper) {
    case "suma": echo suma(2,4);
                  break;
    case "multi": echo multi(2,4);
                  break;
}

// Es conveniente comprobación previa
if (function_exists($oper))
    echo $oper(2,4);

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
43



PHP(1). Funciones
 Funciones anónimas

Funciones anónimas / clausuras / closures / cierres (PHP>=5.3)

No tienen nombre

Función anónima

```

$saludo = function($nombre) {
    echo "Hola ", $nombre;
};
$saludo('Mundo');

$mensaje = 'hola';
$ejemplo = function () {
    var_dump($mensaje); // warning: undefined
};
$ejemplo(); // null

```

Clausura:
función anónima con acceso a ámbito exterior (entorno léxico)

```

$ejemplo = function () use ($mensaje) {
    var_dump($mensaje);
};
$ejemplo(); // hola

$mensaje = 'mundo';
$ejemplo(); // hola

```

```

$mensaje = 'hola';
$ejemplo = function () use (&$mensaje) {
    var_dump($mensaje);
};
$ejemplo(); // hola

$mensaje = 'mundo';
$ejemplo(); // mundo

$ejemplo = function ($arg) use ($mensaje) {
    var_dump($arg . ' ' . $mensaje);
};
$ejemplo("hola"); // hola mundo

```

<https://www.php.net/manual/es/functions.anonymous.php>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
44



PHP(1). Funciones
 Funciones anónimas

Funciones anónimas / clausuras / closures / cierres (PHP>=5.3)

No tienen nombre

```

function crearF($mult) {
    return function($val) use ($mult) {
        return $val*$mult;
    };
}

$f10 = crearF(10);
$f100 = crearF(100);

echo $f10(3), PHP_EOL;
echo $f100(3), PHP_EOL;

```

```

$f1 = function($p) {
    echo "Ejecutando A",PHP_EOL;
    return "Cuando me llamaste p valía $p";
};

function f2($p) {
    echo "Ejecutando B",PHP_EOL;
    $q = $p*2; // q es local a f2 (cuando acabe f2 se destruye)
    return function() use ($q) {
        echo "Ejecutando C",PHP_EOL;
        return "Cuando me creaste q valía $q";
    };
}

$a1 = $f1(2); // Ejecutando A
$b2 = f2(2); // Ejecutando B
echo $a, PHP_EOL; // Cuando me llamaste p valía 2
echo $b(), PHP_EOL; // Ejecutando C
// Cuando me creaste q valía 4

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
45



PHP(1). Funciones
 Funciones flecha

Funciones flecha (arrow functions) (PHP >=7.4)

Son funciones anónimas (con sintaxis simplificada) con cierre del ámbito padre completo. Solo admiten 1 instrucción (expresión que van a devolver).

`fn() => ...`

```

$y = 1;

// Función flecha:
$fn1 = fn($x) => $x + $y;

// Equivalente a esta función anónima (iojo: paso por valor!):
$fn2 = function ($x) use ($y) {
    return $x + $y;
};

echo $fn1(3); // 4
echo $fn2(3); // 4
  
```

<https://www.php.net/manual/es/functions.arrow.php>
 © Javier Martínez Baena



PHP(1). Funciones
 Funciones flecha

Funciones flecha (arrow functions) (PHP >=7.4)

Son funciones anónimas (con sintaxis simplificada) con cierre del ámbito padre completo. Solo admiten 1 instrucción (expresión que van a devolver).

`fn() => ...`

```

function lee() {
    $n = readline("Dime tu nombre: ");
    $msg = "Hola $n, bienvenido/a";
    return fn() => $msg;
}

$a = lee();
$b = lee();

echo $a(), PHP_EOL;
echo $b(), PHP_EOL;
  
```

```

$f1 = fn($p) => "Cuando me llamaste p valía $p";

function f2($p) {
    $q = $p*2;
    return fn() => "Cuando me creaste q valía $q";
}

$a = $f1(2); // Ejecutando A
$b = f2(2); // Ejecutando B
echo $a, PHP_EOL; // Cuando me llamaste p valía 2
echo $b(), PHP_EOL; // Ejecutando C
// Cuando me creaste q valía 4
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



Tecnologías Web
 3º Grado en Ingeniería Informática
PHP (1). Introducción a PHP

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
- » 6. Gestión de errores
7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
 © Javier Martínez Baena



PHP(1). Gestión de errores

Depuración

Depuración de errores

- Configuración de PHP
- Uso de Magic Constants (__LINE__, __FUNCTION__, __FILE__, ...)

```
/* Modificación de la configuración de PHP (php.ini)*/
ini_set("display_errors","1"); /* display_errors = On */
ini_set("error_reporting",E_ALL); /* E_ALL | E_NOTICE | E_WARNING | ... */
```

Directivas accesibles desde PHP: <http://www.php.net/manual/en/ini.list.php>

Uso de PHP-xdebug

Vamos a provocar una división por cero

Fatal error: Uncaught DivisionByZeroError: Division by zero in /home/jbaena/cloud/Docencia/TecnologiasWeb/2324/src/xdebug/xdebugmode01.php on line 8

Call Stack

#	Time	Memory	Function	Location
1	0.0004	361272	{main}()	.../xdebugmode01.php:0
2	0.0004	361272	f()	.../xdebugmode01.php:3
3	0.0004	361272	g()	.../xdebugmode01.php:5

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

49



PHP(1). Gestión de errores

Supresión y generación de errores

Supresión de errores en expresiones

@ antepuesto a una expresión anula el posible mensaje de error

```
/* División por cero: provoca Warning */
echo "Dividiendo por cero ...<br>";
$x = 4 / 0; /* Muestra mensaje
@ $x = 4 / 0; /* No muestra mensaje

/* Acceso a elemento que no existe en array: provoca Notice */
$v['color'] = 'rojo';
echo $v['talla']; /* Muestra mensaje
echo @ $v['talla']; /* No muestra mensaje
```

Generar un error en tiempo de ejecución

Ejemplo de uso: comprobación de parámetros correctos en una función

```
echo "Provocando un error<br>";
trigger_error('Ha habido un error', E_USER_ERROR); // E_USER_NOTICE, E_USER_WARNING
```

Finalizar ejecución de un script

```
die("He acabado"); /* Evitar el uso de die() en sistemas en producción
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

50



PHP(1). Gestión de errores

Manejador de errores

Función manejadora de errores

```
function gestionError($err, $msg, $fi, $li) {
    echo 'Código de error: ', $err, '<br>';
    echo 'Descripción del error: ', $msg, '<br>';
    echo 'Fichero del error: ', $fi, '<br>';
    echo 'Línea del error: ', $li, '<br>';
}

set_error_handler('gestionError');

$x = 4/0;
```

Código de error: 2
 Descripción del error: Division by zero
 Fichero del error: /home/jbaena/Documents/Docencia/TecnologiasWeb/php/debug1.php
 Línea del error: 89

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

51



PHP(1). Gestión de errores
 Excepciones

Excepciones

```

try {
    // Código que genera una excepción de tipo Error
    funcion_inexistente(3);
} catch (Error $e) { // Puede haber varios catch
    echo 'Excepción capturada: ', $e->getMessage(), '<br>';
    echo 'Código: ', $e->getCode(), '<br>';
    echo 'Fichero: ', $e->getFile(), '<br>';
    echo 'Línea: ', $e->getLine(), '<br>';
} finally { // Se ejecuta cuando acaba try o catch
    echo 'Acabado bloque try/catch<br>';
}
  
```

Excepciones predefinidas: <http://php.net/manual/en/reserved.exceptions.php>

Lanzar una excepción:

```
throw new Error('hay un error');
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
52



Tecnologías Web
 3º Grado en Ingeniería Informática
PHP (1). Introducción a PHP

PHP (1). Introducción a PHP

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
- » 7. Características de programación orientada a objetos
8. Apéndice: Expresiones regulares

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena



PHP(1). Características OOP
 Clases, atributos y métodos

Clases, atributos y métodos

```

class Persona {
    var $nombre, $apellidos, $edad;
    function __construct($nombre, $ape, $edad) {
        $this->nombre = $nombre;
        $this->apellidos = $ape;
        $this->edad = $edad;
        $this->normalizaNombre();
    }
    function __destruct() {
        print('... destruyendo objeto'.PHP_EOL);
    }
    function normalizaNombre() {
        $this->nombre = ucfirst(strtolower($this->nombre));
        $this->apellidos = ucfirst(strtolower($this->apellidos));
    }
    function nombreCompleto() {
        return $this->nombre.' '.$this->apellidos;
    }
}
$obj1 = new Persona('JAVIER', 'martinez', '20');
print($obj1->nombreCompleto());
print($obj1->edad);
  
```

→ Atributos

→ Constructor (único)
 function Persona(\$nombre, \$ape, \$edad) (PHP<7)

→ Acceso al objeto: \$this

→ Destructor

→ Métodos

→ Crear objeto

→ Acceso a atributos y métodos

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
54



PHP(1). Características OOP
 Herencia

Herencia (simple)

```

class Trabajador extends Persona {
    var $empresa, $departamento, $categoria;

    function datosEmpresa() {
        return $this->departamento.' ('.$this->empresa.')';
    }

    function nombreCompleto() {
        return $this->nombre.' '.$this->apellidos.' ('.$this->categoria.')';
    }
}

$obj2 = new Trabajador('JAVIER', 'martínez', '20');
$obj2->empresa = 'UGR';
$obj2->departamento = 'CCIA';
$obj2->categoria = 'Prof. Titular';
print($obj2->nombreCompleto()); print(PHP_EOL);
  
```

Nueva clase hereda de otra

Polimorfismo

Si no hay nuevo constructor/destructor se utiliza el de la superclase

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
55



PHP(1). Características OOP
 Herencia

Herencia: constructores

```

class Trabajador extends Persona {
    var $empresa, $departamento, $categoria;

    function __construct($nombre, $ape, $edad) {
        parent::__construct($nombre, $ape, $edad); // Llamada explícita
        $this->empresa='';
        $this->departamento='';
        $this->categoria='';
    }
}
  
```

- Si existe constructor propio **no** se ejecuta el de la superclase
- Si fuese necesario hay que ejecutarlo de forma explícita con `parent::`

`parent::` también se puede usar para métodos de la superclase si hubiese ambigüedad

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
56



PHP(1). Características OOP
 Herencia

Herencia: constructores

```

class Trabajador extends Persona {
    function __construct($nombre, $ape, $edad, $emp, $dep, $cat) {
        parent::__construct($nombre, $ape, $edad); // Llamada explícita
        $this->empresa=$emp;
        $this->departamento=$dep;
        $this->categoria=$cat;
    }
    ...
}

$obj2 = new Trabajador('JAVIER', 'martínez', '20', 'UGR', 'CCIA', 'Prof. Titular');

$obj3 = new Trabajador('JAVIER', 'martínez', '20'); // ERROR
  
```

- El constructor de la subclase puede ser diferente y puede seguir llamando al de la superclase.
- El objeto se crea con su constructor (no con el de la superclase)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
57

PHP(1). Características OOP

Visibilidad de miembros

```

class Persona {
    private $nombre;
    protected $apellidos;
    public $edad;

    public function __construct($nombre, $ape, $edad) {
        $this->nombre = $nombre;
        $this->apellidos = $ape;
        $this->edad = $edad;
        $this->normalizaNombre();
    }
    private function normalizaNombre() {
        $this->nombre = ucfirst(strtolower($this->nombre));
        $this->apellidos = ucfirst(strtolower($this->apellidos));
    }
    ...
}

class Trabajador extends Persona {
    protected $empresa, $departamento, $categoria;
    public function nombreCompleto() {
        //return $this->nombre.' '.$this->apellidos.' ('.$this->categoria.')';
        return parent::nombreCompleto().' ('.$this->categoria.')';
    }
}

```

- **public** (por defecto). Accesible desde dentro y fuera de la clase
- **protected**. Accesible desde dentro de la clase y sus subclases
- **private**. Accesible solo desde dentro de la clase

Las declaraciones con **var** son sinónimo de **public** (no se utilizan)

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 58

PHP(1). Características OOP

Miembros static

Miembros static
 Son globales a todas las instancias de la clase

```

class Persona {
    ...
    private function normalizaNombre() {
        $this->nombre = Persona::norma($this->nombre);
        $this->apellidos = Persona::norma($this->apellidos);
    }
    protected static function norma($cad) {
        return ucfirst(strtolower($cad));
    }
}

class Trabajador extends Persona {
    static $sueldo = ['Asociado'=>'1000','Titular'=>'2000','Catedrático'=>'3000'];
    ...
    public function getSuelo() {
        if (array_key_exists($this->categoria, Trabajador::$sueldo))
            return Trabajador::$sueldo[$this->categoria];
        else
            return '0';
    }
}

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 59

PHP(1). Características OOP

Clases abstractas

Clases abstractas

- No son instanciables
- Pueden incluir métodos abstractos (que deben definirse en la subclase)

```

Abstract class Persona {
    protected $nombre, $apellidos, $edad;
    ...
    abstract public function nombreCompleto(); // No puede incluir cuerpo
}

class Trabajador extends Persona {
    ...
    public function nombreCompleto() { // Obligatorio definir
        return $this->nombre.' '.$this->apellidos.' ('.$this->categoria.')';
    }
}

$objj1 = new Persona('JAVIER','martínez','20'); // No se puede
$objj2 = new Trabajador('JAVIER','martínez','20','UGR','CCIA','Titular');

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 60



PHP(1). Características OOP
 Interfaces

Interfaces

- Definen una interfaz (sin ninguna implementación)
- No contienen atributos
- No se pueden instanciar
- Una clase puede implementar múltiples interfaces

```

Abstract class Persona {
    ...
    abstract public function nombreCompleto(); // No puede incluir cuerpo
}

interface Asalariado {
    function getSuelto();
}

class Trabajador extends Persona implements Asalariado {
    public function nombreCompleto() { // Obligatorio
        return $this->nombre.' '.$this->apellidos.' ('.$this->categoria.')';
    }
    public function getSuelto() { // Obligatorio
        if (array_key_exists($this->categoria, Trabajador::$suelto))
            return Trabajador::$suelto[$this->categoria];
        else
            return '0';
    }
}
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
61



PHP(1). Características OOP
 Traits

Traits (características o rasgos)

- Clases → solo se puede heredar de una.
- Interfaces → se pueden implementar varias pero no permiten incluir código o atributos.

Traits: contenedores que tienen atributos y funcionalidad.

- Una clase puede hacer uso de varios traits.
- No son instanciables.

```

trait Asalariado {
    static $suelto = ['Asociado'=>'1000','Titular'=>'2000','Catedrático'=>'3000'];
    public function getSuelto() {
        if (array_key_exists($this->categoria, Asalariado::$suelto))
            return Asalariado::$suelto[$this->categoria];
        else
            return '0';
    }
}

class Trabajador extends Persona {
    use Asalariado;
}
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena
62



Tecnologías Web
 3º Grado en Ingeniería Informática
PHP (1). Introducción a PHP

1. Introducción
2. Variables y tipos
3. Cadenas
4. Arrays
5. Funciones
6. Gestión de errores
7. Características de programación orientada a objetos
- » 8. Apéndice: Expresiones regulares


Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada
© Javier Martínez Baena

PHP(1). Expresiones regulares

Delimitadores y búsquedas

Uso de expresiones regulares

Buscar coincidencias de patrones
 Sustituir trozos de un texto
 Dividir un texto en trozos

Las expresiones regulares están delimitadas:

- Por un carácter cualquiera
- Al principio y al final

```

/^[a-z]|[0-9]/
~^[a-z]|[0-9]~
X^[a-z]|[0-9]X
([a-z]|[0-9])
{^[a-z]|[0-9]}
  
```

El delimitador también puede ser (), [], {} o <>

Los delimitadores son escapados si forman parte de la expresión:

```

/http://\//
~http://~
  
```

Buscar una coincidencia

```
preg_match(patrn,cadena)
```

Se busca el patrón en la cadena y devuelve true/false dependiendo de si hay emparejamiento o no

```

echo preg_match('/a/', 'Hola'), PHP_EOL; // True
echo preg_match('/x/', 'Hola'), PHP_EOL; // False
echo preg_match('/o!/', 'Hola'), PHP_EOL; // True
echo preg_match('/lo/', 'Hola'), PHP_EOL; // False
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 64

PHP(1). Expresiones regulares

Caracteres especiales y metacaracteres

Caracteres especiales

\d	Cualquier dígito	\D	Cualquier no dígito
\s	Espacio (blanco, tab, enter, ...)	\S	No espacio
\w	Letras, dígitos o _	\W	Ni letras, ni dígitos, ni _
.	Cualquier carácter		

```

echo preg_match('/\d\d\d/', '23x'), PHP_EOL; // True
echo preg_match('/\d\d\d/', '23xabc'), PHP_EOL; // True
echo preg_match('/\D\d\d/', 'x23'), PHP_EOL; // True
echo preg_match('/\D\d\d/', '123'), PHP_EOL; // False
echo preg_match('/x\sx/', 'x x'), PHP_EOL; // True
echo preg_match('/x\sx/', 'x\tx'), PHP_EOL; // True

echo preg_match('/./', 'Hola'), PHP_EOL; // True (3 coincidencias)
echo preg_match('/Ho.a/', 'Hola'), PHP_EOL; // True
echo preg_match('/Ho.a/', 'Hoka'), PHP_EOL; // True
echo preg_match('/.l./', 'Hola'), PHP_EOL; // True
  
```

Metacaracteres

\ ^ \$. [] | () ? * { }
 Tienen significado especial. Para que sean parte del patrón buscado hay que escaparlos

```

echo preg_match('/150$/','150$'), PHP_EOL; // False
echo preg_match('/150\$/', '150$'), PHP_EOL; // True
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 65

PHP(1). Expresiones regulares

Clases de caracteres y alternativas

Clases de caracteres

[] Definen conjuntos de caracteres de un tipo

```

echo preg_match('/P[aeiou]/', 'Hola Pepe'), PHP_EOL; // True
echo preg_match('/P[aeiou]/', 'Hola Lepe'), PHP_EOL; // False
echo preg_match('/c[aeiou]s/', 'la casa es una cosa'), PHP_EOL; // True (2)
echo preg_match('/c[aeiou]s/', 'esto es un caos'), PHP_EOL; // False
  
```

Metacaracteres dentro de []: ^ (negación), - (rango)

```

echo preg_match('/c[^aeiou]p/', 'cup es taza'), PHP_EOL; // False
echo preg_match('/c[^aeiou]p/', 'c3po es un robot'), PHP_EOL; // True
echo preg_match('/c[a-k]p/', 'hay un cepo'), PHP_EOL; // True
echo preg_match('/c[a-k]p/', 'hay una copa'), PHP_EOL; // False
echo preg_match('/[4-9]/', '5th'), PHP_EOL; // True
echo preg_match('/[4-9]/', '2th'), PHP_EOL; // False
echo preg_match('/[a-zA-Z]/', '12345'), PHP_EOL; // False
  
```

Alternativas

| Elegir entre 2 o más alternativas

```

echo preg_match('/Pinto|Valdemoro/', 'Madrid'), PHP_EOL; // False
echo preg_match('/Pinto|Valdemoro/', 'Pintar'), PHP_EOL; // False
echo preg_match('/Pinto|Valdemoro/', 'Valdemoro'), PHP_EOL; // True
  
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 66



PHP(1). Expresiones regulares

Repeticiones

Repeticiones			
?	0 o 1 vez	{n}	n veces
*	0 o más veces	{n,m}	Entre n y m veces (incluidos)
+	1 o más veces	{n,}	Al menos n veces

```

echo preg_match('/Ma+carena/', 'Maaaaacarena'), PHP_EOL; // True
echo preg_match('/Ma+carena/', 'Mcarena'), PHP_EOL; // False
echo preg_match('/Ma?carena/', 'Mcarena'), PHP_EOL; // True
echo preg_match('/Ma?carena/', 'Maacarena'), PHP_EOL; // False
echo preg_match('/Ma*carena/', 'Mcarena'), PHP_EOL; // True
echo preg_match('/Ma*carena/', 'Maaacarena'), PHP_EOL; // True
echo preg_match('/Ma{3}carena/', 'Maaacarena'), PHP_EOL; // True
echo preg_match('/Ma{3}carena/', 'Maaaacarena'), PHP_EOL; // False

echo preg_match('/\+[0-9]{2} [0-9]{3} [0-9]{6}/', '+34 958 123456'), PHP_EOL; // True
echo preg_match('/\+[0-9]{2} [0-9]{3} [0-9]{6}/', '+34 958123456'), PHP_EOL; // False
echo preg_match('/\+[0-9]{2} *[0-9]{3} *[0-9]{6}/', '+34 958123456'), PHP_EOL; // True
echo preg_match('/\(\+[0-9]{2}\) [0-9]{3} [0-9]{6}/', '(+34) 958 123456'), PHP_EOL; // True
echo preg_match('/\(\+[0-9]{2}\) +[0-9]{3} +[0-9]{6}/', '(+34) 958 123456'), PHP_EOL; // True
echo preg_match('/\(\+[0-9]{2}\) \s*[0-9]{3} \s*[0-9]{6}/', '(+34) 958123456'), PHP_EOL; // True
echo preg_match('/\(\+[0-9]{2}\) ?\s*[0-9]{3} \s*[0-9]{6}/', '958123456'), PHP_EOL; // False

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 67



PHP(1). Expresiones regulares

Agrupamientos

Agrupamientos o subpatrones

() Define un grupo dentro de una expresión

```

echo preg_match('/\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}/', '958123456'), PHP_EOL; // False
echo preg_match('/\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}/', '958123456'), PHP_EOL; // True

echo preg_match('/\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}/', 'x958123456x'), PHP_EOL; // True

```

Metacaracteres de inicio/fin de expresión

^ Debe emparejar al comienzo de la cadena
\$ Debe emparejar al final de la cadena

```

echo preg_match('/^\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}$/', '958123456'), PHP_EOL; // True
echo preg_match('/^\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}$/', '(+34) 958 123456'), PHP_EOL; // True
echo preg_match('/^\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}$/', 's(+34) 958 123456'), PHP_EOL; // False
echo preg_match('/^\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}$/', '(+34) 958 123456v'), PHP_EOL; // False

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 68



PHP(1). Expresiones regulares

Extracción de coincidencias

Obtener la(s) coincidencia(s) encontrada(s)

`preg_match(patron, cadena, matches)`

```

preg_match('/c[aeiou]p/', "c3po es un robot", $patron);
print_r($patron); // Array ( [0] => c3p )

preg_match('/c[aeiou]s/', "la casa es una cosa", $patron);
print_r($patron); // Array ( [0] => cas ) ... solo la primera

```

El elemento [0] es la coincidencia completa encontrada
Cada [i] siguiente es la coincidencia de cada agrupamiento

```

preg_match('/^\([+0-9]{2}\)?\s*[0-9]{3}\s*[0-9]{6}$/', '(+34) 958 123456', $patron);
print_r($patron); // Array ( [0] => (+34) 958 123456
// [1] => (+34) )

preg_match('/^\([+0-9]{2}\)?\s*([0-9]{3})\s*([0-9]{6})$/', '(+34) 958 123456', $patron);
print_r($patron); // Array ( [0] => (+34) 958 123456
// [1] => (+34)
// [2] => 958
// [3] => 123456 )

```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada © Javier Martínez Baena 69



PHP(1). Expresiones regulares

Extracción de coincidencias

Obtener la coincidencia encontrada en cada grupo

```
preg_match(patron,cadena,matches)
```

?: permite omitir algunos grupos

```
preg_match('/^(?:\([0-9]{2}\))?\s*([0-9]{3})\s*(?:[0-9]{6})$/', '(+34) 958 123456', $patron);
print_r($patron); // Array ( [0] => (+34) 958 123456
// [1] => 958 )
```

Referencias a patrones previos

```
\1 \2 \3 ...
```

```
preg_match('/^\([0-9]{2}\)\s*([0-9]{3})\s*(\2)$/', '(+34) 958 123456', $patron);
print_r($patron); // No encuentra
```

```
preg_match('/^\([0-9]{2}\)\s*([0-9]{3})\s*(\2)$/', '(+34) 958 958958', $patron);
print_r($patron); // Array ( [0] => (+34) 958 958958
// [1] => (+34)
// [2] => 958
// [3] => 958958 )
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

70



PHP(1). Expresiones regulares

Extracción de coincidencias y sustitución

Obtener todas las coincidencias encontradas

```
preg_match_all(patron,cadena,matches)
```

```
preg_match('/c[aeiou]/','la casa es una cosa',$patron);
print_r($patron); // cas (la primera)
```

```
preg_match_all('/c[aeiou]/','la casa es una cosa',$patron);
print_r($patron); // cas cos (todas)
```

Sustituir coincidencias encontradas

```
preg_replace(patron,sustitución,cadena)
```

```
$result = preg_replace('/\s+/', '-', '(+34) 958 123456'); // (+34)-958-123456
```

```
$nombres = ["Pepito Pérez", "José García", "Ana Martín"];
$nuevos = preg_replace('/(\w)\w* (\w+)/', '\1. \2', $nombres);
print_r($nuevos); // P. Pérez, José García, A. Martín
```

```
$nuevos = preg_replace('/(\w)\w* (\w+)/u', '\1. \2', $nombres);
print_r($nuevos); // P. Pérez, J. García, A. Martín
```

\w = caracteres para formar palabras [0-9a-zA-Z]
u: considerar caracteres unicode

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

71



PHP(1). Expresiones regulares

Separación de cadenas

Separación de cadena por patrones

```
preg_split(patron,cadena)
```

```
$trozos = preg_split('/\s/', 'Antonio Pérez García');
print_r($trozos); // "Antonio", "Pérez", "García"
```

Para obtener la posición de cada trozo separado

```
$trozos = preg_split('/([\s,]+)/', 'Pérez García, Antonio', -1, PREG_SPLIT_OFFSET_CAPTURE);
print_r($trozos); // ["Pérez",0], ["García",7], ["Antonio",16]
```

Para obtener también los separadores

```
$trozos = preg_split('/([\s,]+)/', 'Pérez García, Antonio', -1, PREG_SPLIT_DELIM_CAPTURE);
print_r($trozos); // ["Pérez", " ", "García", " ", "Antonio"]
```

```
$trozos = preg_split('#[+~*]#', '4+2*7/5', -1, PREG_SPLIT_DELIM_CAPTURE);
print_r($trozos);
```

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

72



PHP(1). Expresiones regulares
... comentarios

Expresiones regulares
PHP (versiones antiguas) → POSIX
PHP >= 4.2.0 → Incorpora PCRE (Perl Compatible Regular Expressions)
PHP >= 5.3.0 → POSIX obsoleto
PHP >= 7.3 (2015) → Migración de PCRE a PCRE2

<https://www.pcre.org/>
<https://wiki.php.net/rfc/pcre2-migration>
<https://ayesh.me/Upgrade-PHP-7.3#pcre2>

Webs online para comprobar expresiones regulares
<https://regex101.com/>
<https://regexr.com/>
<https://www.regexpal.com/>

Departamento de Ciencias de la Computación e Inteligencia Artificial - Universidad de Granada

© Javier Martínez Baena

73